

Module 8

Data Wrangling & Preparation

Detailed Student Tutorial with Explanations, Guidance, Examples and Hands-on Practice

Student Learning Guide

How to use this tutorial

Read each concept in sequence. Do not jump directly to the code. First understand the problem, then study the example, run the code, inspect the result, and finally answer the checkpoint questions. The goal is not only to clean a dataset, but to explain why each preparation decision is made.

Source basis: This tutorial follows the order and terminology of the uploaded Module 8 - Data Wrangling & Preparation: preparation fundamentals, cleaning, transformation, feature preparation, integration, pipeline thinking, leakage awareness, AI-assisted thinking, and hands-on preparation tasks.

Learning Objectives

By the end of this module, you should be able to:

- Explain why data preparation is necessary before machine learning.
- Distinguish raw data from processed and ML-ready data.
- Identify missing values, duplicates, invalid entries, outliers, and consistency problems.
- Apply suitable cleaning techniques using Pandas.
- Explain and apply scaling, normalization, encoding, and standardization.
- Create, select, and remove features with model readiness in mind.
- Combine datasets using merging and aggregation.
- Design a repeatable preprocessing workflow or pipeline.
- Recognize common forms of data leakage and avoid invalid evaluation.
- Use AI to support preprocessing decisions while validating its suggestions.
- Document a complete data preparation process.

Module Roadmap

Section	Main Focus
1	Data Preparation Fundamentals
2	Data Cleaning
3	Data Transformation
4	Feature Preparation
5	Data Integration
6	Data Pipeline Thinking
7	Data Leakage Awareness
8	Thinking with AI
9	Hands-on Tasks and Mini Project

Key idea

A machine-learning model learns from the data it receives. Data preparation improves the quality, consistency, and representation of that data before model training.

Running Example Dataset

We will reuse the following small customer dataset throughout the tutorial so that every concept connects to the same practical example.

```
import pandas as pd
import numpy as np

data = {
    "Customer_ID": [101, 102, 103, 103, 105, 106, 107, 108],
    "Name": ["Akhil", "Meera", "Rahul", "Rahul", "Sneha", "Arjun", "Niya", "Farhan"],
    "Age": [23, 27, np.nan, 31, 250, 29, 26, 35],
    "Gender": ["Male", "Female", "M", "M", "female", "Male", "F", "male"],
    "Salary": [25000, 32000, 28000, 28000, 45000, np.nan, 30000, 50000],
    "City": ["Kochi", "Calicut", "Kochi", "Kochi", "Trivandrum", "Kochi", None, "Calicut"]
}
```

```
df = pd.DataFrame(data)
print(df)
```

Before changing anything, try to identify the visible problems yourself.

- A missing Age value is present.
- A missing Salary value is present.
- A missing City value is present.
- Customer_ID 103 appears more than once.
- Age 250 is suspicious for a human customer.
- Gender contains inconsistent representations such as Male, male, M, Female, female, and F.

Student habit

Always inspect first. Cleaning without inspection can remove useful information or introduce new errors.

1. Understand Data Preparation Fundamentals

1.1 Why Data Preparation Matters

Real-world data is usually collected for business operations, forms, transactions, devices, websites, or databases. It is not necessarily collected in the exact format required by a machine-learning model. Preparation bridges this gap.

Common reasons preparation is required include:

- Some values are missing.
- Some records are repeated.
- Some entries are impossible or invalid.
- Text categories use inconsistent spelling or capitalization.
- Numerical variables have very different scales.
- Some columns are not useful for the prediction problem.
- Information may be distributed across multiple datasets.

Think about it

A model does not automatically understand that 'M', 'Male', and 'male' should be treated as the same category. The data preparation step must make such meaning explicit.

Guided example

Suppose you want to predict whether a customer will purchase a membership. If salary is missing, gender is inconsistent, and duplicate customers are present, the model may learn a distorted pattern. Data preparation aims to reduce these avoidable problems before training.

Checkpoint

1. Why is a dataset that loads successfully in Pandas not automatically ML-ready?
2. Give three examples of data-quality problems.
3. Explain the phrase 'garbage in, garbage out' in the context of machine learning.

1.2 Raw Data vs Processed Data

Raw Data	Processed Data
Collected in original form	Prepared for analysis/modeling
May contain missing values	Missing values have an intentional handling strategy
May contain inconsistent labels	Categories are made consistent
May contain invalid or duplicate records	Quality problems are investigated and treated
May contain unencoded text	Features are represented in a suitable model-ready form

Processing should preserve useful meaning. The purpose is not to randomly change values; it is to make the dataset more reliable and usable.

Example

```
# Raw values
print(df["Gender"].unique())

# Possible cleaned representation
df["Gender"] = df["Gender"].str.strip().str.lower()
```

```
df["Gender"] = df["Gender"].replace({"m": "male", "f": "female"})
print(df["Gender"].unique())
```

1.3 Data Quality Challenges

Challenge	Example	Why It Matters
Missing value	Salary = NaN	Model or analysis may fail or become biased
Duplicate	Same customer repeated	Repeated observations can distort statistics
Invalid entry	Age = 250	Model may learn from impossible values
Inconsistency	Male / male / M	Same meaning may be treated as different categories
Outlier	Salary much higher than most	Can strongly affect some statistics or algorithms

Important

An outlier is not automatically an error. It is an unusual value that should be investigated.

1.4 What Makes a Dataset ML-Ready?

An ML-ready dataset is not defined by one universal checklist, but it should normally be clean enough, consistent enough, and represented appropriately for the selected algorithm.

- Relevant features are selected.
- Target and features are clearly separated.
- Missing values have a defined strategy.
- Data types are correct.
- Categorical variables are encoded appropriately when required.
- Numerical transformations are applied when appropriate.
- Training and evaluation data are kept separate to avoid leakage.

```
print(df.info())
print(df.isnull().sum())
print(df.duplicated().sum())
print(df.describe(include="all"))
```

1.5 Data Preparation Lifecycle

A simple preparation lifecycle is:

4. Collect or ingest the data.
5. Inspect the dataset and understand columns.
6. Identify quality problems.
7. Clean missing, duplicate, invalid, and inconsistent data.
8. Transform features into useful representations.
9. Create or select relevant features.
10. Integrate additional data sources when needed.
11. Validate the prepared dataset.
12. Split data correctly for model training and evaluation.
13. Apply repeatable preprocessing through a pipeline when possible.

Guidance

Do not treat data preparation as a one-time command. It is a sequence of decisions that should be reproducible and explainable.

2. Understand Data Cleaning

2.1 Missing Values

A missing value means the expected information is unavailable. In Pandas, missing numerical values commonly appear as NaN, while Python objects may contain None.

Detect missing values

```
print(df.isnull())
print(df.isnull().sum())
```

Common handling strategies

Strategy	Example	Use with Care When
Drop rows	<code>df.dropna()</code>	The dataset is small or missingness is large
Mean imputation	<code>fillna(mean)</code>	Outliers make the mean unrepresentative
Median imputation	<code>fillna(median)</code>	Useful for skewed numerical data, but still an estimate
Mode imputation	<code>fillna(mode)</code>	The most common category may overrepresent one group
Special category	Unknown	Missingness itself may carry meaning

Example: median for Age

```
median_age = df["Age"].median()
df["Age"] = df["Age"].fillna(median_age)
```

Example: mode for City

```
mode_city = df["City"].mode()[0]
df["City"] = df["City"].fillna(mode_city)
```

Guidance

Do not memorize 'fill every NaN with mean'. First ask: Why is the value missing? How much is missing? Is the feature numerical or categorical? Would an estimate distort the dataset?

Practice

14. Find the percentage of missing values in each column.
15. Compare mean and median Salary. Which would you choose if one salary were extremely large?
16. Create a copy of the dataset and test row deletion. How many rows remain?

2.2 Duplicate Records

A duplicate occurs when the same logical observation appears more than once. However, identical-looking rows are not always errors. For example, two identical transactions may represent two genuine purchases.

```
print(df.duplicated())
print("Full-row duplicates:", df.duplicated().sum())

# Inspect repeated customer identifiers
print(df[df.duplicated(subset=["Customer_ID"], keep=False)])
```

If you confirm that a record is truly duplicated:

```
df = df.drop_duplicates()
```

Guidance

Before removing duplicates, define the unit of observation. Is one row a customer, a payment, a visit, or a transaction? The answer changes what 'duplicate' means.

2.3 Invalid Entries

Invalid entries violate expected rules or domain constraints. Examples include negative prices, impossible ages, malformed dates, or unsupported category values.

```
# Identify suspicious ages
print(df[df["Age"] > 100])

# Mark suspicious ages as missing for later treatment
df.loc[df["Age"] > 100, "Age"] = np.nan
df["Age"] = df["Age"].fillna(df["Age"].median())
```

Validation rules to consider

- Range rules: Age should be within an expected range.
- Type rules: Salary should be numeric if it represents money.
- Category rules: Status should belong to an allowed set.
- Date rules: End date should not occur before start date without a valid reason.
- Uniqueness rules: An identifier may need to be unique.

2.4 Outlier Identification

An outlier is a value that lies unusually far from the rest of the observed values. Detection helps you investigate whether it is a genuine rare case, a data-entry error, or a value that needs a special treatment.

IQR method

The interquartile range is calculated as $IQR = Q3 - Q1$. A common rule flags values below $Q1 - 1.5 \cdot IQR$ or above $Q3 + 1.5 \cdot IQR$.

```
Q1 = df["Salary"].quantile(0.25)
Q3 = df["Salary"].quantile(0.75)
IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

outliers = df[(df["Salary"] < lower) | (df["Salary"] > upper)]
print(outliers)
```

Do not do this automatically

Do not delete every outlier just because a formula flags it. Investigate whether the value is legitimate and whether the selected model is sensitive to extreme values.

2.5 Data Consistency Improvements

Consistency means that equivalent values use a common representation.

```
df["Gender"] = df["Gender"].str.strip().str.lower()
df["Gender"] = df["Gender"].replace({
    "m": "male",
```



```
    "f": "female"  
  })  
  print(df["Gender"].value_counts())
```

Other consistency examples

- Dates: 31-08-2026 versus 2026/08/31.
- Units: kilograms versus pounds.
- Currency: INR versus USD.
- Text capitalization: Kochi versus kochi.
- Whitespace: 'Calicut ' versus 'Calicut'.

Section 2 Review Questions

17. What is the difference between missing and invalid data?
18. Why can duplicate removal be dangerous if you do not understand what one row represents?
19. Why is median sometimes preferred over mean for skewed numerical data?
20. Is every outlier an error? Explain.
21. Give two examples of data consistency problems.

3. Understand Data Transformation

Data transformation changes how a feature is represented. The original information is retained as much as possible, but the representation is adjusted to support analysis or machine learning.

3.1 Feature Transformation

A transformation can change the distribution, scale, or mathematical form of a feature. For example, a heavily skewed positive variable such as income may sometimes be transformed using a logarithm.

```
df["Log_Salary"] = np.log1p(df["Salary"])
print(df[["Salary", "Log_Salary"]].head())
```

Guidance

Use transformations because you understand a reason for them - not because they appear in a checklist.

3.2 Scaling

Scaling changes numerical magnitudes so features become more comparable. This is especially important for algorithms that depend on distance or magnitude.

Min-Max scaling

A common formula is: $x_{\text{scaled}} = (x - x_{\text{min}}) / (x_{\text{max}} - x_{\text{min}})$. The observed minimum maps to 0 and maximum to 1.

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
scaled = scaler.fit_transform(df[["Age", "Salary"]])
print(scaled[:5])
```

Why scaling can matter

If Age ranges from 20 to 60 while Salary ranges from 20,000 to 100,000, a distance-based algorithm may be dominated by Salary simply because its numbers are larger.

3.3 Normalization

In introductory preprocessing discussions, normalization often refers to rescaling values into a fixed range such as 0 to 1. In other contexts, normalization can also mean adjusting vectors to unit length. Always confirm which meaning is intended in the tool or lesson.

Student note

Do not use 'normalization' and 'standardization' as if they are always identical. They are related preprocessing ideas with different calculations.

3.4 Encoding Categorical Variables

Many ML algorithms need numerical input, so categories such as City or Gender often need encoding.

Binary mapping

```
df["Gender_Code"] = df["Gender"].map({
    "female": 0,
    "male": 1
})
```

One-hot encoding

```
city_encoded = pd.get_dummies(df["City"], prefix="City", dtype=int)
print(city_encoded.head())
```

One-hot encoding creates a separate indicator column for each category. This avoids introducing an artificial order such as Kochi=1, Calicut=2, Trivandrum=3.

Guidance

Before encoding, clean the category labels. Otherwise 'Male', 'male', and 'M' may incorrectly become separate encoded categories.

3.5 Data Standardization

Standardization commonly transforms a value into the number of standard deviations it lies from the mean: $z = (x - \text{mean}) / \text{standard_deviation}$.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
standardized = scaler.fit_transform(df[["Age", "Salary"]])
print(standardized[:5])
```

Technique	Typical Result	Main Idea
Min-Max scaling	Often 0 to 1	Rescale using minimum and maximum
Standardization	Mean approximately 0, standard deviation approximately 1	Center and scale using mean and standard deviation
One-hot encoding	0/1 columns	Represent nominal categories without artificial ranking

Section 3 Practice

22. Apply Min-Max scaling to Age and Salary.
23. Apply StandardScaler to the same features and compare the output.
24. One-hot encode City.
25. Explain why using City=1,2,3 may be misleading for nominal categories.
26. Write one sentence explaining when scaling can affect a distance-based model.

4. Understand Feature Preparation

4.1 Feature Creation

Feature creation means deriving new useful variables from existing data. The goal is to make relevant information easier for a model to use.

Examples

- Date of Birth -> Age
- Joining Date -> Membership Duration
- Quantity and Unit Price -> Total Amount
- Transaction history -> Number of Purchases
- Salary -> Annual Salary

```
df["Annual_Salary"] = df["Salary"] * 12
print(df[["Salary", "Annual_Salary"]].head())
```

Guidance

A created feature should have a meaningful relationship to the problem. Feature engineering is not random column multiplication.

4.2 Feature Selection Awareness

Feature selection means deciding which available inputs are useful for the prediction task.

For a purchase-prediction example, Customer_ID may identify a row without describing customer behaviour. Name may also be irrelevant. Age, Salary, City, and previous behavior may be more meaningful.

Questions to ask before keeping a feature

- Does this feature describe information available at prediction time?
- Is it related to the problem?
- Is it only an identifier?
- Does it duplicate another feature?
- Could it introduce leakage?
- Is its quality sufficient?

4.3 Removing Irrelevant Information

```
df_model = df.drop(columns=["Name"], errors="ignore")
```

Guidance

Do not remove a column merely because it looks inconvenient. Understand its business meaning first.

4.4 Improving Model Readiness

```
print(df.info())
print(df.isnull().sum())
print(df.duplicated().sum())
print(df.describe(include="all"))
```

Before training, confirm that each feature has a clear meaning and a valid representation.

4.5 Data Quality Improvements

Quality Dimension	Question
Accuracy	Does the value correctly represent reality?
Completeness	Are required values present?
Consistency	Are equivalent values represented the same way?
Validity	Does the value follow required rules?
Uniqueness	Are records duplicated when they should be unique?

5. Understand Data Integration

5.1 Combining Datasets

In real systems, information may be split across customer, payment, attendance, product, or transaction tables. Data integration combines useful information into a dataset suitable for analysis.

5.2 Data Merging

```
customers = pd.DataFrame({
    "Customer_ID": [101, 102, 103],
    "Name": ["Akhil", "Meera", "Rahul"]
})

payments = pd.DataFrame({
    "Customer_ID": [101, 102, 103],
    "Amount": [5000, 7000, 4500]
})

merged = pd.merge(customers, payments, on="Customer_ID", how="inner")
print(merged)
```

Understand the join key

The merge works because Customer_ID represents the same entity in both datasets. Before merging, verify that the key has compatible data types and meaning.

5.3 Data Aggregation

Aggregation summarizes multiple rows into higher-level information.

```
transactions = pd.DataFrame({
    "Customer_ID": [101, 101, 102, 103, 103, 103],
    "Amount": [1000, 2000, 4000, 500, 700, 900]
})

summary = transactions.groupby("Customer_ID").agg(
    Total_Spent=("Amount", "sum"),
    Average_Spent=("Amount", "mean"),
    Purchase_Count=("Amount", "count")
).reset_index()

print(summary)
```

This converts a transaction-level table into customer-level features.

5.4 Working with Multiple Sources

- CSV or Excel files
- SQL databases
- APIs
- CRM systems
- Website or mobile app logs
- Cloud data platforms

5.5 Dataset Consistency

Before integration, check:

- Do both datasets use the same ID format?

- Do numerical columns use the same units?
- Do dates follow compatible formats?
- Do category labels mean the same thing?
- Does a column with the same name actually represent the same concept?

Example warning

If one Salary column means monthly salary and another means annual salary, simply merging them under the same name creates incorrect data.

6. Understand Data Pipeline Thinking

6.1 Data Ingestion

Data ingestion is the process of bringing data from its source into the processing environment.

```
df_csv = pd.read_csv("customers.csv")  
# Other sources may include Excel, SQL databases, APIs, or cloud storage.
```

6.2 Data Cleaning Workflows

Instead of applying random commands, follow a repeatable order:

27. Load data.
28. Inspect schema and basic statistics.
29. Measure missing values.
30. Check duplicates.
31. Validate ranges and categories.
32. Investigate outliers.
33. Standardize representations.
34. Re-check data quality.

6.3 Transformation Workflows

Numerical and categorical features may need different preparation steps. A common idea is to impute missing numerical values before scaling and to clean categorical values before encoding.

6.4 Reusable Processing Pipelines

```
from sklearn.pipeline import Pipeline  
from sklearn.impute import SimpleImputer  
from sklearn.preprocessing import StandardScaler  
  
numeric_pipeline = Pipeline([  
    ("imputer", SimpleImputer(strategy="median")),  
    ("scaler", StandardScaler())  
])
```

A pipeline records the order of preprocessing steps so they can be applied consistently.

6.5 Automation Awareness

In a production workflow, new data can be automatically validated, cleaned, transformed, and passed to a model. Automation reduces repeated manual work and improves consistency.

Guidance

A reusable pipeline is valuable because the same rules that prepare training data can later prepare new incoming data.

7. Understand Data Leakage Awareness

7.1 What Is Data Leakage?

Data leakage occurs when the training process receives information that would not legitimately be available when the model makes a real prediction.

Simple example

If you want to predict whether a student will pass an exam, but include the final exam result as an input feature, the model already has information that reveals the answer.

Memory aid

Leakage is like seeing the answer before taking the exam. The score may look excellent, but it does not prove true predictive ability.

7.2 Common Leakage Scenarios

- A target-related outcome is accidentally included as a feature.
- Information created after the event being predicted is included.
- Preprocessing is fitted using the full dataset before train/test separation.
- Duplicate or closely related records appear in both training and test sets.

Preprocessing leakage example

```
# Risky pattern:
# scaler.fit_transform(X) before the train/test split

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

The scaler learns parameters from the training data only. The test set is transformed using those learned parameters.

7.3 Avoiding Invalid Datasets

Maintain a clear boundary between information used to learn preprocessing/model parameters and information reserved for final evaluation.

7.4 Reliable ML Workflows

Pipelines help make preprocessing repeatable and reduce the risk of accidentally applying different steps to training and evaluation data.

7.5 Validation Awareness

A very high evaluation score is not automatically trustworthy. Ask whether the evaluation setup reflects the information that will be available in the real-world prediction scenario.

Section 7 Checkpoint

35. Define data leakage in your own words.
36. Give one example of a feature that could leak the target.
37. Why should a scaler normally be fitted on training data rather than the complete dataset?
38. Why can an unusually high test score be a warning sign?

8. Thinking with AI

The module encourages using AI for cleaning suggestions, dataset analysis, and transformation recommendations, while validating generated insights and preprocessing decisions.

8.1 AI for Data Cleaning Suggestions

Example prompt for a learning assistant:

Prompt

My dataset has Age, Salary, Gender, and City. Age has missing values, Gender contains Male/male/M/Female/F, and one Age is 250. Suggest cleaning options, explain the risks of each option, and tell me what I should inspect before changing the data.

8.2 AI for Dataset Analysis

AI can help generate questions you should investigate, such as unusual ranges, missingness patterns, or inconsistent categories. It should not replace direct inspection of the dataset.

8.3 AI for Transformation Recommendations

Prompt

I want to use KNN with Age, Salary, City, and Gender. Which preprocessing steps should I investigate, and why?

8.4 Validate Generated Insights

Do not apply an AI recommendation just because it sounds confident. Verify it against:

- The actual dataset.
- The meaning of each feature.
- The selected algorithm.
- The prediction-time scenario.
- The risk of leakage.

8.5 Verify Preprocessing Decisions

A good student should be able to answer: What did I change? Why did I change it? What alternatives existed? What risk did the decision introduce?

Best learning use of AI

Ask AI to explain and challenge your reasoning, not merely to provide code to copy.

9. Apply Through Hands-on Tasks

The final part of the module is practical: clean messy data, handle missing values, transform raw data, prepare an ML-ready dataset, build preprocessing pipelines, and document decisions.

9.1 Mini Project Dataset

```
import pandas as pd
import numpy as np

data = {
    "Customer_ID": [1, 2, 3, 3, 5, 6, 7, 8, 9, 10],
    "Age": [22, 25, np.nan, 30, 150, 29, 33, 41, 27, 35],
    "Gender": ["Male", "Female", "M", "M", "female", "Male", "F", "Female", "male", "Female"],
    "Salary": [25000, 35000, 30000, 30000, 45000, np.nan, 52000, 60000, 28000, 48000],
    "City": ["Kochi", "Calicut", "Kochi", "Kochi", "Trivandrum", None, "Calicut", "Kochi", "Trivandrum", "Calicut"]
}

df = pd.DataFrame(data)
print(df)
```

9.2 Task 1 - Inspect Before Cleaning

```
print(df.head())
print(df.shape)
print(df.info())
print(df.describe(include="all"))
print(df.isnull().sum())
```

Write down every problem you observe before modifying the DataFrame.

9.3 Task 2 - Handle Missing Values

```
df["Age"] = df["Age"].fillna(df["Age"].median())
df["Salary"] = df["Salary"].fillna(df["Salary"].median())
df["City"] = df["City"].fillna(df["City"].mode()[0])
```

Guidance: Explain why you used median for numerical variables and mode for the category. This is a learning example; in a real project, you should investigate missingness before selecting an imputation strategy.

9.4 Task 3 - Investigate Duplicates

```
print(df[df.duplicated(subset=["Customer_ID"], keep=False)])
print("Exact duplicates:", df.duplicated().sum())
```

Decide whether the repeated customer record is truly duplicated. Document the reasoning before deleting anything.

9.5 Task 4 - Treat Invalid Age

```
df.loc[df["Age"] > 100, "Age"] = np.nan
df["Age"] = df["Age"].fillna(df["Age"].median())
```

9.6 Task 5 - Standardize Gender

```
df["Gender"] = df["Gender"].str.strip().str.lower()
df["Gender"] = df["Gender"].replace({"m": "male", "f": "female"})
print(df["Gender"].value_counts())
```

9.7 Task 6 - Encode Categorical Variables

```
df["Gender"] = df["Gender"].map({"female": 0, "male": 1})
df = pd.get_dummies(df, columns=["City"], dtype=int)
print(df.head())
```

9.8 Task 7 - Separate Features and Target

```
y = df["Purchased"]
X = df.drop(columns=["Purchased", "Customer_ID"])
```

The target is Purchased. Customer_ID is removed from this example because it is being treated as an identifier rather than a behavioural feature.

9.9 Task 8 - Split Before Fitting Preprocessing

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

9.10 Task 9 - Standardize Age and Salary

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train[["Age", "Salary"]] = scaler.fit_transform(
    X_train[["Age", "Salary"]]
)

X_test[["Age", "Salary"]] = scaler.transform(
    X_test[["Age", "Salary"]]
)
```

Critical question

Why do we use fit_transform on the training data but only transform on the test data? Because preprocessing parameters should be learned from training data, not from the held-out test set.

9.11 Task 10 - Final Validation

```
print(X_train.info())
print(X_train.isnull().sum())
print(X_train.head())
print(y_train.head())
```

Confirm that the prepared feature matrix has the expected columns, no unintended missing values, and appropriate numerical representations.

10. Guided Preprocessing Pipeline Example

The module introduces reusable processing pipelines. The following example shows how numerical and categorical preparation can be organized more systematically.

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

numeric_features = ["Age", "Salary"]
categorical_features = ["Gender", "City"]

numeric_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer([
    ("num", numeric_pipeline, numeric_features),
    ("cat", categorical_pipeline, categorical_features)
])
```

This pipeline does not replace reasoning. It records the chosen reasoning in a reusable form.

How to Read the Pipeline

- 39. `numeric_features` identifies numerical columns.
- 40. `numeric_pipeline` first fills missing numerical values using the median.
- 41. The numerical values are then standardized.
- 42. `categorical_pipeline` fills missing categories with the most frequent category.
- 43. `OneHotEncoder` converts categories into numerical indicator columns.
- 44. `ColumnTransformer` applies each pipeline only to its intended columns.

Guidance

Fit the preprocessing object on training data. Reuse the fitted object to transform validation, test, and future incoming data.

11. How to Document Preparation Decisions

The module asks students to document preparation decisions. A professional preparation note should record what was found, what was changed, and why.

Problem Found	Decision	Reason	Risk / Note
Missing Age	Median imputation	Avoid losing rows in learning example	Imputed value is an estimate
Age > 100	Mark as invalid, then impute	Value is outside chosen expected range	Domain rule should be verified
Gender variants	Standardize labels	Same meaning had multiple spellings	Mapping must reflect true categories

City category	One-hot encode	Nominal categories have no natural numeric order	New/unseen categories require handling
Age/Salary scale	Standardize	Prepare for scale-sensitive models	Not every model requires scaling

Student Preparation Log Template

- Dataset name:
- Prediction objective:
- Target column:
- Features considered:
- Missing-value findings:
- Duplicate findings:
- Invalid-entry rules:
- Outlier investigation:
- Consistency changes:
- Encoding decisions:
- Scaling/standardization decisions:
- Features created or removed:
- Integration steps:
- Leakage checks:
- Final validation results:

12. Student Exercises

Exercise A - Concept Check

45. Define data preparation.
46. What is the difference between raw and processed data?
47. List five common data-quality challenges.
48. What does ML-ready mean?
49. Explain why missing-value treatment depends on context.
50. What is a duplicate record?
51. What is an invalid entry?
52. What is an outlier?
53. What is categorical encoding?
54. What is standardization?
55. What is feature creation?
56. What is data integration?
57. What is aggregation?
58. What is a preprocessing pipeline?
59. What is data leakage?

Exercise B - Code Tasks

60. Create a DataFrame containing at least 12 rows and intentionally add two missing values.
61. Add one duplicate row and one invalid numerical value.
62. Use `isnull().sum()` to measure missing data.
63. Use `duplicated()` to inspect repeated rows.
64. Correct inconsistent category labels using `str.strip()`, `str.lower()`, and `replace()`.
65. Impute a numerical column using the median.
66. Impute a categorical column using the mode.
67. Use the IQR method to flag potential outliers.
68. One-hot encode one nominal categorical column.
69. Standardize two numerical columns.
70. Create one new meaningful feature.
71. Separate features and target.
72. Split the dataset into training and test sets.
73. Fit preprocessing on the training data and transform the test data.
74. Write a short preparation report explaining every decision.

Exercise C - Reasoning Questions

75. A salary value is 10 times larger than every other salary. Should you remove it? What should you investigate first?
76. A dataset contains gender values M, Male, male, F, Female, and female. What should happen before encoding?
77. A student-pass model includes `Final_Result` as an input. What problem does this create?
78. Why can a customer ID be unsuitable as a predictive feature even though it is numerical?
79. Why can fitting `StandardScaler` before train/test splitting create leakage?
80. Why does a pipeline improve reproducibility?
81. How could two datasets with a column called Salary still be inconsistent?

13. Mini Project Assignment

Prepare a messy dataset for machine learning and submit both the final cleaned dataset and a preparation report.

Required Steps

82. Load or create a dataset with at least 30 rows.
83. Describe each column.
84. Identify missing values.
85. Identify duplicates.
86. Define at least two validation rules for invalid entries.
87. Investigate possible outliers.
88. Standardize inconsistent text categories.
89. Encode categorical data where appropriate.
90. Scale or standardize suitable numerical features.
91. Create at least one meaningful feature.
92. Remove at least one irrelevant feature if justified.
93. Split data into train and test sets correctly.
94. Explain how you avoided data leakage.
95. Build a reusable preprocessing workflow or pipeline.
96. Document every major preparation decision.

Suggested Evaluation Rubric

Area	Marks
Understanding and inspection	15
Data cleaning	20
Transformation and encoding	20
Feature preparation	15
Pipeline and leakage awareness	15
Documentation and explanation	15

14. Quick Reference Cheat Sheet

Task	Pandas / Scikit-learn Example
Check missing values	<code>df.isnull().sum()</code>
Drop missing rows	<code>df.dropna()</code>
Fill numerical missing values	<code>df['Age'].fillna(df['Age'].median())</code>
Find duplicates	<code>df.duplicated().sum()</code>
Remove exact duplicates	<code>df.drop_duplicates()</code>
Inspect unique categories	<code>df['City'].unique()</code>
Standardize text	<code>df['City'].str.strip().str.lower()</code>
Replace category values	<code>df['Gender'].replace({'...'})</code>
One-hot encode	<code>pd.get_dummies(df, columns=['City'])</code>
Min-Max scale	<code>MinMaxScaler()</code>
Standardize	<code>StandardScaler()</code>
Merge tables	<code>pd.merge(left, right, on='ID')</code>
Aggregate	<code>df.groupby('ID').agg(...)</code>
Split data	<code>train_test_split(...)</code>
Build pipeline	<code>Pipeline([...])</code>

Preparation Flow to Remember

RAW DATA -> INSPECT -> CLEAN -> TRANSFORM -> PREPARE FEATURES -> INTEGRATE -> SPLIT -> FIT PREPROCESSING ON TRAINING DATA -> TRANSFORM -> VALIDATE -> MODEL

Final reminder

The purpose of data wrangling is not to make the dataset look neat. The purpose is to make preprocessing decisions that are justified, consistent, reproducible, and suitable for reliable machine learning.

15. Glossary

Term	Meaning
Data wrangling	Organizing, cleaning, transforming, and preparing raw data for use.
Imputation	Replacing missing values using a defined strategy.
Outlier	An observation unusually different from most others.
Encoding	Converting categorical information to a numerical representation.
Scaling	Adjusting numerical magnitudes to a comparable scale.
Standardization	Centering and scaling using mean and standard deviation.
Feature	An input variable used by a model.
Target	The outcome the model is intended to predict.

Aggregation	Summarizing multiple rows into a higher-level value.
Pipeline	A repeatable sequence of preprocessing/modeling steps.
Data leakage	Improper access to information that would not be available at prediction time.

16. Final Self-Assessment

Before considering this module complete, you should be able to answer YES to the following:

- I can explain why data preparation matters.
- I can inspect a dataset before changing it.
- I can choose a missing-value strategy and explain my reasoning.
- I can distinguish duplicates, invalid values, and outliers.
- I can standardize inconsistent categories.
- I understand the purpose of scaling, normalization, encoding, and standardization.
- I can create and select useful features.
- I can merge and aggregate data.
- I can explain a preprocessing pipeline.
- I can identify basic data leakage scenarios.
- I can use AI suggestions critically rather than blindly.
- I can document my preprocessing decisions.